

Detailed Report: Code Analysis of 2_task.R

July 10, 2026

1 Introduction

This report provides a detailed, line-by-line analysis of the 2_task.R file, which corresponds to Task 2 of the project. The primary objective of this script is to perform a Posterior Predictive Check. It utilizes the best Beta mixture model selected in Task 1 to generate the posterior predictive density and compares it visually against the empirical distribution (histogram) of the observed data. Furthermore, it visualizes the individual weighted components of the mixture.

2 Line-by-Line Code Analysis

2.1 Initialization and Grid Setup

```
1 # =====
2 # TASK 2: Posterior Predictive Check
3 # =====
4
5 cat("\nExecuting Task 2: Generating Posterior Predictive Check...\n")
6
7 # Extract MCMC samples from the best mode
8 samples_mat <- as.matrix(mcmc_samples)
9 S <- nrow(samples_mat)
10
11 # Define a grid of values for y (Load factor is between 10^-3 and 1 - 10^-3)
12 y_grid <- seq(0.001, 0.999, length.out = 500)
13 pred_density <- rep(0, length(y_grid))
```

- **Lines 1-5:** Introductory comments and a console message indicating the start of the task.
- **Lines 7-9:** The MCMC samples from the best model (saved as `mcmc_samples` in Task 1) are extracted and converted into a matrix format. `S` stores the total number of MCMC samples (iterations $\times$ chains) available.
- **Lines 11-13:** A grid of 500 equally spaced values, `y_grid`, is defined between 0.001 and 0.999. Since the data represents a load factor modeled with a Beta distribution, its support must strictly be in the interval (0,1). The vector `pred_density` is initialized with zeros to store the aggregated overall predictive density across this grid.

2.2 Computing the Posterior Predictive Density

```
15 # We need to average the predictive density over all MCMC samples
16 cat("Computing posterior predictive density over MCMC samples...\n")
17
18 # Matrix to store the densities of individual components
19 comp_density <- matrix(0, nrow = length(y_grid), ncol = best_model)
20
21 for (s in 1:S) {
22   # For each MCMC sample, compute the mixture density on the grid
23   mix_dens_s <- rep(0, length(y_grid))
24   for (h in 1:best_model) {
25     pi_h <- samples_mat[s, paste0("pi[", h, "]")]
26     alpha_h <- samples_mat[s, paste0("alpha[", h, "]")]
27     beta_h <- samples_mat[s, paste0("beta[", h, "]")]
```

```

28
29   # Add weighted Beta density for component h
30   comp_dens_h <- pi_h * dbeta(y_grid, alpha_h, beta_h)
31   mix_dens_s <- mix_dens_s + comp_dens_h
32   comp_density[, h] <- comp_density[, h] + comp_dens_h
33 }
34 # Accumulate
35 pred_density <- pred_density + mix_dens_s
36 }
37
38 # Average over S samples to get the final Bayesian predictive density
39 pred_density <- pred_density / S
40 comp_density <- comp_density / S

```

- **Lines 18-19:** A matrix `comp_density` is initialized to store the cumulative densities for each individual component of the mixture. The number of rows equals the grid size, and the number of columns corresponds to the number of components in the best model (`best_model`).
- **Line 21:** An outer loop begins, iterating over each MCMC sample `s` from 1 to `S`.
- **Line 23:** `mix_dens_s` is initialized to zero. This vector will hold the overall mixture density evaluated on the grid for the specific sample `s`.
- **Line 24:** An inner loop iterates through each component `h` of the selected best mixture model.
- **Lines 25-27:** For the current sample `s` and component `h`, the parameters π_h (weight), α_h , and β_h are extracted from the MCMC matrix.
- **Lines 29-32:** The weighted Beta density for component `h` is computed over the entire `y_grid`. This density (`comp_dens_h`) is added to the current sample's mixture density (`mix_dens_s`) and is also accumulated in the respective column of the `comp_density` matrix.
- **Lines 34-36:** The computed mixture density for sample `s` is accumulated into the global `pred_density` vector.
- **Lines 38-40:** The accumulated `pred_density` and `comp_density` are divided by the total number of samples `S`. This computes the Monte Carlo approximation of the posterior predictive density (by integrating out the parameters over their posterior distribution).

2.3 Data Formatting for Visualization

```

42 # Create a data frame for plotting the curve
43 df_pred <- data.frame(
44   y_grid = y_grid,
45   density = pred_density
46 )
47
48 # Convert component density matrix to a long format data frame for ggplot
49 df_comp <- as.data.frame(comp_density)
50 colnames(df_comp) <- paste0("Component_", 1:best_model)
51 df_comp$y_grid <- y_grid
52 df_comp_long <- pivot_longer(df_comp, cols = starts_with("Component"), names_to = "
    Component", values_to = "density")

```

- **Lines 42-46:** The averaged overall predictive density is stored in a data frame `df_pred`, which will be used for plotting the main curve.
- **Lines 48-52:** The `comp_density` matrix is converted into a data frame `df_comp`, and descriptive column names are assigned. Then, using the `pivot_longer` function (from the `tidyr` package), the data frame is transformed from a wide format to a long format (`df_comp_long`). This long format is strictly required by `ggplot2` to map the different components to a color aesthetic easily.

2.4 Plotting the Posterior Predictive Check

```
54 # Plotting using ggplot2
55 cat("Generating plots...\n")
56
57 # Posterior predictive density
58 p_predictive <- ggplot(df, aes(x = load_factor)) +
59   geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "tomato", color = "
   white", alpha = 0.8) +
60   geom_line(data = df_pred, aes(x = y_grid, y = density), color = "darkblue", linewidth
   = 1.2) +
61   theme_minimal() +
62   labs(
63     title = paste("Posterior Predictive Density vs Empirical Histogram (H =", best_model
   , ")"),
64     x = "Load factor (hourly / daily peak)",
65     y = "Density"
66   ) +
67   theme(
68     plot.title = element_text(face = "bold", size = 14),
69     axis.title = element_text(face = "bold")
70   )
71
72 # Display the overall plot
73 print(p_predictive)
74 ggsave("images/posterior_predictive.png", plot = p_predictive, width = 8, height = 5,
   dpi = 300)
75 cat("\nPlot complessivo salvato come 'images/posterior_predictive.png'.\n")
```

- **Lines 57-72:** A ggplot2 object `p_predictive` is built. It overlays the empirical histogram of the original data (where the y-axis is scaled to density using `after_stat(density)` so that the area integrates to 1) with the estimated posterior predictive density curve. The original data is assumed to be stored in the `df` data frame (likely initialized in `0_setup.R`).
- **Lines 74-77:** The plot is displayed in the console/viewer and saved to the disk at `images/posterior_predictive.png` with high resolution (300 dpi).

2.5 Plotting the Individual Components

```
79 # Only singles component graphic
80 p_components <- ggplot() +
81   geom_line(data = df_comp_long, aes(x = y_grid, y = density, color = Component),
   linewidth = 1) +
82   theme_minimal() +
83   labs(
84     title = paste("Individual Mixture Components (H =", best_model, ")"),
85     x = "Load factor",
86     y = "Density (Weighted)",
87     color = "Component"
88   ) +
89   theme(
90     plot.title = element_text(face = "bold", size = 14),
91     axis.title = element_text(face = "bold")
92   )
93
94 # Display the components plot
95 print(p_components)
96 ggsave("images/components_only.png", plot = p_components, width = 8, height = 5, dpi =
   300)
97 cat("Plot dei singoli componenti salvato come 'images/components_only.png'.\n")
```

- **Lines 79-92:** A second plot, `p_components`, is created using the long-format data frame `df_comp_long`. This plot displays the individual weighted density curves for each component in the mixture model.
- **Lines 94-98:** This second plot is displayed and saved to `images/components_only.png`.

3 Conclusion

This script successfully implements a rigorous Bayesian posterior predictive check. Rather than using point estimates (like the posterior mean) for the model parameters, it correctly averages the predictive density across all MCMC samples. This approach incorporates parameter uncertainty into the predictive distribution. The visual comparison provided by the generated `ggplot2` graphs allows for an immediate assessment of how well the selected mixture model captures the empirical distribution of the load factor data.